

TP1 – Gestion de l'équipage

4PYTH – Projet fil rouge « Fondation » ·

Mercredi 7 octobre, 15h15 – 16h45



Contexte

Dans un futur lointain, la galaxie est plongée dans une crise sans précédent. La Seconde Fondation, un groupe secret maîtrisant les sciences mentales, lutte pour maintenir la stabilité galactique. Mais un chef rebelle connu sous le nom du **Mulet** menace cet équilibre.

Officier de la flotte marchande indépendante, vous êtes chargé de réorganiser les équipages de vos vaisseaux pour préparer une contre-offensive. Première mission : **gérer les membres d'équipage** et vous assurer que vos vaisseaux sont opérationnels avant l'arrivée des renforts. 🙌

Ce TP en bref

- **Objectifs** : manipuler une liste de dictionnaires, écrire des fonctions, valider les saisies de l'utilisateur et construire un menu interactif.
- **À terminer pour le vendredi 9 octobre**, début du TP2. Vous pouvez avancer chez vous.
- Ce TP est la première brique du **projet Fondation**, remis le 5 novembre. Commitez et poussez **régulièrement** sur GitHub.
- L'IA est autorisée pour le projet, à une condition : **comprendre chaque ligne** de votre code. Je le vérifierai en soutenance.



Le dépôt GitHub

≈ 10 min

Git · venv

Tout le projet vit dans **un seul dépôt**, que vous enrichirez de TP en TP.

1. Créez un dossier de projet, son environnement virtuel et initialisez Git :

```
Terminal

mkdir 4pyth-fondation
cd 4pyth-fondation
python3 -m venv .venv      # Windows : py -m venv .venv
git init
```

2. Créez un fichier `.gitignore`, pour que l'environnement virtuel ne soit jamais envoyé sur GitHub :

```
.gitignore

.venv/
__pycache__/
```

3. Créez un fichier `README.md` avec au moins le titre du projet et votre nom. Vous le complétez au fil des TP (mode d'emploi, bonus réalisés, déclaration d'usage de l'IA).
4. Sur GitHub, créez un dépôt privé nommé `4pyth-fondation-nom-prenom`, sans README, puis reliez-le et poussez :

```
Terminal

git add .
git commit -m "Initialisation du projet Fondation"
git branch -M main
git remote add origin https://github.com/votre-pseudo/4pyth-fondation-nom-prenom.git
git push -u origin main
```

5. Dans **Settings** → **Collaborators**, invitez **Chris-CHEVALIER**, au plus tard le vendredi 9 octobre.

Premier commit de votre vie ? Git vous demande votre identité



```
git config --global user.name "Prénom Nom"  
git config --global user.email "adresse@exemple.fr"
```

Utilisez l'adresse de votre compte GitHub.

1 L'équipage

≈ 10 min

Listes · dictionnaires

L'équipage est une **liste** de membres. Chaque membre est un **dictionnaire** contenant cinq clés :

Clé	Contenu	Type
<code>first_name</code>	prénom	<code>str</code>
<code>last_name</code>	nom de famille	<code>str</code>
<code>gender</code>	<code>"F"</code> ou <code>"M"</code>	<code>str</code>
<code>age</code>	âge en années	<code>int</code>
<code>role</code>	un rôle de la liste autorisée (voir étape 2)	<code>str</code>

Dans un fichier `main.py`, créez l'équipage de départ en copiant ces cinq membres :

```
main.py – équipage de départ

crew = [
    {"first_name": "Bel", "last_name": "Riose", "gender": "M", "age": 48, "role": "commandant"},
    {"first_name": "Gaal", "last_name": "Dornick", "gender": "F", "age": 34, "role": "technicien"},
    {"first_name": "Hugo", "last_name": "Crast", "gender": "M", "age": 37, "role": "pilote"},
    {"first_name": "Salvor", "last_name": "Hardin", "gender": "F", "age": 28, "role": "armurier"},
    {"first_name": "Novi", "last_name": "Sura", "gender": "F", "age": 25, "role": "entretien"},
]
```

Le reste de la flotte vous attend dans le jeu de données ci-dessous. Vous ajouterez ces membres **avec votre propre fonction** `add_member` : c'est la meilleure façon de la tester.

Jeu de données – flotte « Galactica », commandant Bel Riose

Nom	Genre	Âge	Rôle
Bel Riose	M	48	commandant
Gaal Dornick	F	34	technicien
Salvor Hardin	F	28	armurier
Hugo Crast	M	37	pilote
Jagger Fount	M	41	pilote
Hober Mallow	M	52	pilote
Lathan Devers	M	46	marchand
Limmar Ponyets	M	43	marchand
Novi Sura	F	25	entretien
Ebling Mis	M	34	entretien
Eskel Gorov	M	32	marchand

Nom	Genre	Âge	Rôle
Fallom	F	28	pilote
Han Pritcher	M	31	armurier
Harlan Branno	F	38	entretien
Lewis Pirenne	M	34	pilote
Munn Compor	M	27	technicien
Yugo Amary	M	33	technicien
Bayta Darell	F	48	entretien
Toran Darell	M	52	technicien
Stora Gendibal	F	35	technicien
Jord Commasson	M	53	entretien

Trois mentalistes complètent la flotte : Tellem Bond (M, 48 ans), Yanna Seldon (F, 52 ans) et Joie (F, 25 ans). Ils ont des pouvoirs particuliers : vous les modéliserez au TP2.

2

Les fonctions de gestion

≈ 50 min

Fonctions · conditions · boucles · exceptions

Dans un fichier `crew.py`, écrivez les quatre fonctions suivantes. Chacune reçoit l'équipage en paramètre.

`add_member(crew)` – Ajouter un membre

1. Demande le prénom, le nom de famille, le genre, l'âge et le rôle du nouveau membre avec `input()`.
2. Refuse toute saisie invalide et redemande tant qu'elle n'est pas correcte :
 - prénom et nom de famille : **entre 3 et 15 caractères** ;
 - nom de famille **unique** dans l'équipage, sans tenir compte de la casse ;
 - genre : `"F"` ou `"M"` ;
 - âge : un **nombre entier**. Une saisie comme `vingt` ne doit pas faire planter le programme ;
 - rôle : uniquement une valeur de la liste `ROLES`, sans tenir compte de la casse.
3. Ajoute le dictionnaire du nouveau membre à la liste et affiche une confirmation.

crew.py – en haut du fichier

```
ROLES = ["commandant", "pilote", "technicien", "armurier", "marchand", "entretien"]
```

Réflexe cyber : la liste blanche

Plutôt que d'essayer d'interdire les mauvaises valeurs (liste noire, toujours incomplète), on **n'accepte que les valeurs connues**. C'est le principe de validation des entrées recommandé partout en sécurité : formulaires web, paramètres d'API, règles de pare-feu...

`remove_member(crew)` – Retirer un membre

1. Demande le nom de famille du membre à retirer.
2. Le retire de la liste et affiche une confirmation.
3. Affiche un message clair si aucun membre ne porte ce nom.

`display_crew(crew)` – Afficher l'équipage

1. Affiche chaque membre sur une ligne, numérotée.
2. Affiche un message si l'équipage est vide.

`check_crew(crew)` – Vérifier l'équipage

1. Vérifie que l'équipage compte **au moins 2 membres**, dont **au moins un pilote et un technicien**.
2. Affiche `L'équipage est prêt pour la mission !` ou, sinon, ce qui manque.
3. Renvoie `True` ou `False` : le TP2 en aura besoin.

Indices

- **Une fonction, une tâche** : une petite fonction `ask_int(prompt)` qui redemande un entier jusqu'à obtenir une saisie valide vous servira pour l'âge... et pour le menu de l'étape 3.
- Même idée pour la recherche : une fonction `find_member(crew, last_name)` qui renvoie le membre ou `None` sert à la fois pour l'ajout (unicité) et pour la suppression.
- Revoyez les diapos « `try` , `except` , `else` , `raise` » et « `while` , `break` , `continue` » du cours.



Le menu interactif

≈ 20 min

while · match / case · modules

1. Dans `main.py`, importez vos fonctions depuis `crew.py`.
2. Affichez le menu dans une boucle `while True` et traitez le choix avec `match / case`.
3. L'option **[0] Quitter** sort de la boucle. Les TP suivants ajouteront les options [5], [6], etc.
4. Un choix invalide (`abc`, `7` ...) affiche un message et **ne fait jamais planter** le programme.
5. Placez le lancement du menu sous `if __name__ == "__main__":`.

```
Proposition d'affichage

===== Flotte marchande – Gestion de l'équipage =====
[1] Ajouter un membre
[2] Retirer un membre
[3] Afficher l'équipage
[4] Vérifier l'équipage
[0] Quitter
Votre choix : 3
  1. Bel Riose (M, 48 ans) – commandant
  2. Gaal Dornick (F, 34 ans) – technicien
  3. Hugo Crast (M, 37 ans) – pilote
  4. Salvor Hardin (F, 28 ans) – armurier
  5. Novi Sura (F, 25 ans) – entretien
```

Commitez !

Un commit par étape terminée, avec un message qui dit ce qui a changé : `git add .` puis `git commit -m "Ajout du menu interactif"` et `git push`.



Pour aller plus loin

Facultatif

Bonus 1 – Terminal de bord sécurisé

- Au lancement, demandez un **code d'accès** avant d'afficher le menu. Masquez la saisie avec `getpass`.
- Après **3 codes incorrects**, le terminal se verrouille et le programme s'arrête.
- Votre code d'accès est écrit en clair dans votre code source... ce qui est une mauvaise pratique. En production, on ne stocke que son empreinte, calculée par une fonction dédiée comme `bcrypt` ou `argon2`.

Bonus 2 – Contraintes d'âge

- Un pilote doit avoir au moins 25 ans, un technicien au moins 18 ans.
- Aucun membre ne peut avoir plus de 65 ans.

Bonus 3 – Promotions

- Ajoutez une clé `experience` (en années) aux membres.
- Un technicien ayant au moins 10 ans d'expérience peut être **promu pilote**, si les contraintes d'âge sont respectées.

Bonus 4 – Rapport d'équipage

- Ajoutez une option de menu qui affiche le nombre total de membres, la répartition par rôle et les membres proches de l'âge maximum.

À méditer

Essayez d'ajouter **Toran Darell** puis **Bayta Darell** et enfin **Fallom**, qui n'a pas de nom de famille. Vos règles de validation fonctionnent... mais sont-elles adaptées à la réalité ? Une règle trop stricte bloque des utilisateurs légitimes ; une règle trop souple laisse passer n'importe quoi. Trouver le bon équilibre est tout l'enjeu de la validation des entrées.



Avant de rendre

Vérifiez chaque point. Ils correspondent à la grille d'évaluation du projet Fondation.

- ☐ Les quatre options du menu fonctionnent et [0] quitte proprement.
- ☐ Aucune saisie ne fait planter le programme : lettres à la place d'un nombre, option inexistante, nom inconnu...
- ☐ Les règles de validation sont respectées : longueur des noms, nom unique, genre, rôle de la liste autorisée.
- ☐ Les messages guident l'utilisateur : il sait toujours ce qui s'est passé et ce qu'il doit faire.
- ☐ Le code est réparti entre `crew.py` (fonctions) et `main.py` (menu).
- ☐ Noms de variables et de fonctions en `snake_case` et en anglais, avec une docstring par fonction.
- ☐ Le dépôt est sur GitHub, j'y suis invité et il contient plusieurs commits.
- ☐ Vous pouvez expliquer **chaque ligne** de votre code.